DAAS API 0.21.7

Reference Manual

March 17, 2026

1 Data Structure Documentation

1.1 DaasAPI Class Reference

Public Member Functions

- const char ∗ getVersion ()
- const char ∗ getBuildInfo ()
- const char ∗ listAvailableDrivers ()
- daas_error_t doEnd ()
- daas_error_t doReset ()
- daas_error_t doInit (din_t sid, din_t din)
- daas_error_t doPerform (performs_mode_t mode)
- daas_error_t enableDriver (link_t driver_id, const char ∗local_uri)
- nodestate_t getStatus ()
- void setAcceptRequestsLevel (accept_request_policy_t policy_level)
- bool storeConfiguration (IDepot ∗storage_interface)
- bool loadConfiguration (IDepot ∗storage_interface)
- bool doStatisticsReset ()
- uint64_t getSystemStatistics (syscode_t label)
- daas_error_t map (din_t din)
- daas_error_t map (din_t din, link_t link, const char ∗suri)
- daas_error_t map (din_t din, link_t link, const char ∗suri, const char ∗skey)
- daas_error_t remove (din_t din)
- daas_error_t discovery ()
- daas_error_t discovery (link_t link)
- daas_error_t discovery (din_t sid)
- void setDiscoveryState (discovery_state_t mode)
- dinlist_t listNodes ()
- daas_error_t locate (din_t din, int timeout=1000)
- daas_error_t sendStatus (din_t din)
- const nodestate_t & status (din_t din)
- const nodestate_t & fetch (din_t din, uint16_t opts)
- uint64_t getSyncedTimestamp ()
- const nodestate_t & unlock (din_t din, const char ∗skey)
- const nodestate_t & lock (const char ∗skey, unsigned policy_)

- const nodestate_t & syncNode (din_t din, unsigned timezone)
- const nodestate_t & syncNet (din_t din, unsigned bubble_time)
- void setATSMaxError (int32_t error)
- bool use (din_t din, uint32_t rx_queue_size=0, uint32_t retry=3, uint32_t timeout=1000)
- bool end (din_t din)
- uint32_t send (din_t din, uint8_t *outbound, uint32_t size, uint32_t timeout=1000, uint32_t retry=3)
- uint32_t received (din_t din)
- uint32_t receive (din_t din, uint8_t *&inbound)
- tsetlist_t & listTypesets ()
- daas_error_t pull (din_t din, DDO **inboundDDO)
- daas_error_t push (din_t din, DDO *outboundDDO)
- daas_error_t availablesPull (din_t din, uint32_t &count)
- daas_error_t addTypeset (const uint16_t typeset_code, const typeset_fun fun)
- daas_error_t frisbee (din_t din)
- daas_error_t frisbeeICMP (din_t din, uint32_t timeout, uint32_t retry)
- daas_error_t frisbeeDPERF (din_t din, uint32_t sender_pkt_total=10, uint32_t block_size=1024 *1024, uint32_t sender_trip_period=0)
- dperf_info_result getFrisbeeResultDPERF ()
- daas_error_t setDDOPolicy (ddo_policy_t policy)
- daas_error_t unbindNetwork ()
- daas_error_t setOptions (option_t option, uint32_t val)
- din_t getSid ()
- din_t getDin ()
- const char * errorToString (daas_error_t error_code)

1.1.1 Member Function Documentation

addTypeset()

```
daas_error_t addTypeset (
            const uint16_t typeset_code,
            const typeset_fun fun)
```

Maps a user-defined typeset to a handler function. If a DDO with the specified typeset is received, the provided handler function will be called.

Parameters

typeset_code the code of the typeset to add

fun The function to handle received DDOs of this typeset

> **Note**
> The registered typeset does not trigger the received DDO event.

Returns

ERROR_NONE on success, or an error code on failure.

Example:

```
#define hello_worlds_typeset 120
#define random_calculation_typeset 122

node.doPerform(PERFORM_CORE_THREAD);

node.addTypeset(hello_worlds_typeset, [](din_t din){
    DDO *pk;
    if (node.pull(din, &pk) == ERROR_NONE){
        unsigned char * payload = pk->getPayloadPtr();
        printf("Hello World from DIN %llu: %s\n", din, payload);
        delete pk;
    }
});

node.addTypeset(random_calculation_typeset, [](din_t din){
    DDO *pk;
    if (node.pull(din, &pk) == ERROR_NONE){
        unsigned char * payload = pk->getPayloadPtr();
        int limit = *((int*)payload);
        int random_number = rand() % limit;
        printf("Random number from DIN %llu with limit %d: %d\n", din, limit, random_number);
        delete pk;
    }
});

/*
    Sender side:

    DDO hello_ddo;
    hello_ddo.setTypeset(hello_worlds_typeset);
    node.push(remoteDIN, &hello_ddo);

    On the receiving side, the handler for hello_worlds_typeset will be triggered and print
  the message.
*/
```

availablesPull()

```
daas_error_t availablesPull (
            din_t din,
            uint32_t & count)
```

Checks if there are available DDOs from a remote node.

Parameters

din DIN of the remote node to check for available DDOs

count reference to a variable that will hold the number of available DDOs

Returns

ERROR_NONE on success, or an error code on failure.

See also

pull

discovery() [1/3]

```
daas_error_t discovery ()
```

Starts a discovery process to locate nodes in the network.
This process will send the enabled drivers URI to a sort of broadcast in order to auto-map this node to remote ones.

> **Note**
> The reception of this packet cannot be controlled, it can only be disabled.

Parameters

none : starts discovery on all available links

> **Note**
> If the node is already connected to a network, it will automatically disconnect from the current network if it discovers a new one during the discovery process.

Returns

ERROR_NONE on success, or an error code on failure.

> **See also**
> doInit

Example:

```
// In case of missing initialization, the node can generate din and sid automatically if
  discovery is used.
// The sid will be assigned by the node master of the network we are connecting to.
// The din will be generated automatically by the node itself and will be combined with some
  time releated value given by the
// network we are connecting to.

if (node.enableDriver(_LINK_INET4, "127.0.0.1:2020") == ERROR_NONE)
{
    fprintf(stdout, "Driver enabled successfully.\n");
}
else
{
    fprintf(stderr, "Failed to enable driver.\n");
    return -1;
}

node.doPerform(PERFORM_CORE_THREAD);

// Send discovery beacons for 10 times with 100ms interval
for (int i = 0; i < 10; i++){
    node.discovery();
    sleep(100); // Sleep 100ms between each beacon sent
}

// it also possible to specify the link to use for discovery, in this case the discovery will
  be only trasmitted though inet4 which is the only driver enabled.
node.discovery(_LINK_INET4);

// or the sid of the network to connect to
node.discovery(123); // discovery for network with sid 123

// other logic down here..

// The discovery in this case will be only trasmitted though inet4 which is the only driver
  enabled.
```

discovery() [2/3]

```
daas_error_t discovery (
            din_t sid)
```

Starts a discovery process to locate nodes in the network.
This process will send the enabled drivers URI to a sort of broadcast in order to auto-map this node to remote ones.

> **Note**
> The reception of this packet cannot be controlled, it can only be disabled.

Parameters

sid	The SID of the network to connect to.

Returns

ERROR_NONE on success, or an error code on failure.

> **See also**
> doInit

Example:

```
// In case of missing initialization, the node can generate din and sid automatically if
  discovery is used.
// The sid will be assigned by the node master of the network we are connecting to.
// The din will be generated automatically by the node itself and will be combined with some
  time releated value given by the
// network we are connecting to.

if (node.enableDriver(_LINK_INET4, "127.0.0.1:2020") == ERROR_NONE)
{
    fprintf(stdout, "Driver enabled successfully.\n");
}
else
{
    fprintf(stderr, "Failed to enable driver.\n");
    return -1;
}

node.doPerform(PERFORM_CORE_THREAD);

// Send discovery beacons for 10 times with 100ms interval
for (int i = 0; i < 10; i++){
    node.discovery();
    sleep(100); // Sleep 100ms between each beacon sent
}

// it also possible to specify the link to use for discovery, in this case the discovery will
  be only trasmitted though inet4 which is the only driver enabled.
node.discovery(_LINK_INET4);

// or the sid of the network to connect to
node.discovery(123); // discovery for network with sid 123

// other logic down here..

// The discovery in this case will be only trasmitted though inet4 which is the only driver
  enabled.
```

discovery() [3/3]

```
daas_error_t discovery (
            link_t link)
```

Starts a discovery process to locate nodes in the network.
This process will send the enabled drivers URI to a sort of broadcast in order to auto-map this node to remote ones.

> **Note**
> The reception of this packet cannot be controlled, it can only be disabled.

Parameters

link The communication technology to use for discovery. If not specified, all available links will be used.

Returns

ERROR_NONE on success, or an error code on failure.

> **See also**
> doInit

Example:

```
// In case of missing initialization, the node can generate din and sid automatically if
  discovery is used.
// The sid will be assigned by the node master of the network we are connecting to.
// The din will be generated automatically by the node itself and will be combined with some
  time releated value given by the
// network we are connecting to.

if (node.enableDriver(_LINK_INET4, "127.0.0.1:2020") == ERROR_NONE)
{
    fprintf(stdout, "Driver enabled successfully.\n");
}
else
{
    fprintf(stderr, "Failed to enable driver.\n");
    return -1;
}

node.doPerform(PERFORM_CORE_THREAD);

// Send discovery beacons for 10 times with 100ms interval
for (int i = 0; i < 10; i++){
    node.discovery();
    sleep(100); // Sleep 100ms between each beacon sent
}

// it also possible to specify the link to use for discovery, in this case the discovery will
  be only trasmitted though inet4 which is the only driver enabled.
node.discovery(_LINK_INET4);

// or the sid of the network to connect to
node.discovery(123); // discovery for network with sid 123

// other logic down here..

// The discovery in this case will be only trasmitted though inet4 which is the only driver
  enabled.
```

doEnd()

```
daas_error_t doEnd ()
```

Releases resources and deactivates the node.

Parameters

none

Returns

ERROR_NONE on success, or an error code on failure.

doInit()

```
daas_error_t doInit (
            din_t sid,
            din_t din)
```

Initializes services and resources for the local node.

Parameters

sid SID of the local node

din DIN of the local node

Returns

ERROR_NONE on success, or an error code on failure.

Example:

```
#include <stdio.h>
#include <daas.hpp>

bool exit = false;
DaasAPI node = DaasAPI();

int main() {
    din_t localSID = 100;
    din_t localDIN = 200;

    din_t remoteDIN = 201;

    /*  Note that the initialization of the node is not strictly necessary anymore.
     *  In case of missing din and/or sid, the node can generate those informations.
     *  The sid will be assigned by the node master of the network we are connecting to.
     *  The din will be generated automatically by the node itself and will be combined with some
     *  time releated value given by the network we are connecting to.
     *
     *  However, if the initialization is not performed, the node will not be able to communicate
     *  until it connects to a network through the discovery feature.
     */

    if (node.doInit(localSID, localDIN) == ERROR_NONE)
    {
        fprintf(stdout, "Node initialized with SID: %llu, DIN: %llu\n", localSID, localDIN);
    }
    else
    {
        fprintf(stderr, "Failed to initialize node with SID: %llu, DIN: %llu\n", localSID,
      localDIN);
        return -1;
    }

    if (node.enableDriver(_LINK_INET4, "127.0.0.1:2020") == ERROR_NONE)
    {
        fprintf(stdout, "Driver enabled successfully.\n");
    }
    else
    {
        fprintf(stderr, "Failed to enable driver.\n");
        return -1;
    }

    if (node.map(remoteDIN, _LINK_INET4, "127.0.0.1:2021")) {
        fprintf(stdout, "Remote DIN %u mapped successfully.\n", remoteDIN);
    } else {
        fprintf(stderr, "Failed to map remote DIN %u.\n", remoteDIN);
        return -1;
    }

    node.doPerform(PERFORM_CORE_THREAD);

    while (!exit) {
       // Perform the operation here
    }

    return 0;
}
```

doPerform()

```
daas_error_t doPerform (
            performs_mode_t mode)
```

Executes the node's main processing loop in either real-time or multi-threading mode. It manages the node's internal state, including network communication, data processing, and synchronization with other nodes. In **multi-threading** mode, it spawns internal threads to handle processing.

Parameters

mode

- **PERFORM_CORE_THREAD** for multi-threading mode
- **PERFORM_CORE_NO_THREAD** for real-time mode

Returns

ERROR_NONE on success, or an error code on failure.

> **Note**
> This function **must** be called if the node has to be operational. In **multi-threading** mode it has to be called only once, while in **real-time** mode it has to be called cyclically.

> **See also**
> doInit

doReset()

```
daas_error_t doReset ()
```

Resets the local node and clears all resources. It disconnects also from the network that it currently is.

Parameters

none

Returns

ERROR_NONE on success, or an error code on failure.

doStatisticsReset()

```
bool doStatisticsReset ()
```

Resets the system's statistics data.

Parameters

none

Returns

true if the reset was successful, false otherwise.

enableDriver()

```
daas_error_t enableDriver (
            link_t driver_id,
            const char * local_uri)
```

Configure driver for network technology (links)

Parameters

driver_id the communication technology to enable:
- **_LINK_INET4**
- **_LINK_BLUETOOTH**
- **and** others as defined in link_t

local_uri the physical address of the local node:
- For **_LINK_INET4:** a text string in the format "IP:PORT" (e.g., "192.168.1.2:4000")
- For **_LINK_BLUETOOTH:** a text string representing the Bluetooth address (e.g., "00:1A:7D:DA:71:13:CHANNEL")

Returns

ERROR_NONE on success, or an error code on failure.

See also
doInit

end()

```
bool end (
            din_t din)
```

Ends a real-time session with a remote node.

Parameters

din DIN of the remote node to end the session with

Returns

true if the RT session was successfully ended, false otherwise.

See also
use

errorToString()

```
const char * errorToString (
            daas_error_t error_code)
```

Converts a DaaS error code to a human-readable string.

Parameters

error_code The DaaS error code to convert.

Returns

A string describing the error code.

fetch()

```
const nodestate_t & fetch (
            din_t din,
            uint16_t opts)
```

Fetches the status of a remote node and updates its status in the local instance.

Parameters

din DIN of the remote node to fetch the status from

opts options for fetching

Returns

the nodestate_t of the remote node after fetching.

frisbee()

```
daas_error_t frisbee (
            din_t din)
```

Pings a remote node to check its availability.

Parameters

din DIN of the remote node to ping

Returns

ERROR_NONE on success, or an error code on failure.

frisbeeDPERF()

```
daas_error_t frisbeeDPERF (
            din_t din,
            uint32_t sender_pkt_total = 10,
            uint32_t block_size = 1024 *1024,
            uint32_t sender_trip_period = 0)
```

Measures the data transfer performance toward a remote node by sending a sequence of test packets and evaluating the communication behavior.

Parameters

din	DIN of the remote node to be tested.
sender_pkt↩_total	Total number of packets to send.
block_size	Size of each packet, expressed in bytes.
sender_trip↩_period	Time interval between consecutive packet transmissions, expressed in milliseconds.

Returns

ERROR_NONE on success, or an error code on failure.

frisbeeICMP()

```
daas_error_t frisbeeICMP (
             din_t din,
             uint32_t timeout,
             uint32_t retry)
```

Pings a remote node with a specified timeout and retry count.

Parameters

din	DIN of the remote node to ping
timeout	maximum time to wait for a reply (in milliseconds)
retry	number of retries if no reply is received

Returns

ERROR_NONE on success, or an error code on failure.

getBuildInfo()

```
const char * getBuildInfo ()
```

Retrieves build information of the DaaS API library.

Parameters

none

Returns

A string containing build information such as compiler version and build configuration.

getDin()

```
din_t getDin ()
```

Retrieves the DIN of the local node.

Returns

The DIN of the local node.

getFrisbeeResultDPERF()

```
dperf_info_result getFrisbeeResultDPERF ()
```

Returns the result of the frisbeeDPERF test.

Parameters

none

Returns

a **dperf_info_result** structure containing the performance test results:

- **sender_first_timestamp:** timestamp of the first packet sent by the sender
- **local_end_timestamp:** timestamp of the last packet received by the sender
- **remote_first_timestamp:** timestamp of the first packet received by the receiver
- **remote_last_timestamp:** timestamp of the last packet received by the receiver
- **remote_pkt_counter:** number of packets received by the receiver
- **remote_data_counter:** total data received by the receiver in bytes

getSid()

```
din_t getSid ()
```

Retrieves the SID of the local node.

Returns

The SID of the local node.

getStatus()

```
nodestate_t getStatus ()
```

Returns the status of the local node.

Parameters

none

Returns

the status of the local node.

- This includes hardware version, linked channels, synchronization status, security policy, and more.

getSyncedTimestamp()

```
uint64_t getSyncedTimestamp ()
```

Time with ATS correction used to be able to communicate inside the DaaS network.

Parameters

none

Attention
This function returns the synchronized timestamp of the local node, only when connected to a DaaS network and ATS is in sync. If the node is not synchronized, the function returns the local system time without ATS correction.

Returns

the synced timestamp of the local node.

getSystemStatistics()

```
uint64_t getSystemStatistics (
            syscode_t label)
```

Returns the statistics of the local node for a specific system code.

Parameters

label the system code to get statistics for (e.g., _cor_dme_sended)

Returns

the system statistics for the given label.

getVersion()

```
const char * getVersion ()
```

Retrieves the version of the DaaS API library.

Parameters

none

Returns

A string representing the version of the DaaS API library.

listAvailableDrivers()

```
const char * listAvailableDrivers ()
```

Lists all available drivers in the DaaS stack for this node.

Parameters

none

Returns

A semicolon-separated string containing the list of available drivers.

listNodes()

```
dinlist_t listNodes ()
```

Returns map entries (known nodes) in the local instance.

Parameters

none

Returns

a list of known nodes (din_t) in the local instance.

listTypesets()

```
tsetlist_t & listTypesets ()
```

Returns a list of user-defined typesets.

Parameters

none

Returns

a reference to the list of user-defined typesets.

See also

addTypeset

Note

The list is of type tsetlist_t, which is a Vector of typeset

loadConfiguration()

```
bool loadConfiguration (
            IDepot * storage_interface)
```

Loads the configuration from the specified storage interface.

Parameters

storage_interface	the interface to load the configuration from (e.g., a file system, database, etc.)

Returns

true if the configuration was loaded successfully, false otherwise.

Example:

```
// Simple implementation of IDepot for demonstration purposes
class device_storage : public IDepot
{
    // Open the partition handler
    bool open(bool read_mode)
    {
        if (f == nullptr)
        {
            const char *mode = read_mode ? "rb" : "wb";
            f = fopen("storage.txt", mode);
        }

        return f != nullptr;
    }

    // Close the partition handler
    bool close()
    {

        if (f && fclose(f) == 0)
            f = nullptr;

        return f == nullptr;
    }

    // Deletes all the record saved
    bool clearSpace() {}

    // Get how much space is left on the device
    bool getSpaceInfo() {}

    // Trash a specific entry
    bool trash(unsigned oKey) {}

    // Save map to buffer
    unsigned save(unsigned oKey, unsigned char *pdata, unsigned dsize)
    {
        if (f)
            return fwrite(pdata, dsize, 1, f);

        return 0;
    }

    // Load map data from buffer
    unsigned load(unsigned oKey, unsigned char *pdata, unsigned dsize)
    {
        if (f)
            return fread(pdata, dsize, 1, f);

        return 0;
    }

private:
    FILE *f = nullptr;
};

bool exit = false;
device_storage storage;
DaasAPI node = DaasAPI();

int main() {

    // load configuration from storage
    node.loadConfiguration(&storage);

    node.doPerform(PERFORM_CORE_THREAD);

    // Perfom operation here

    // To store configuration
    node.storeConfiguration(&storage);


    return 0;
}
```

locate()

```
daas_error_t locate (
            din_t din,
            int timeout = 1000)
```

It starts a process to locate the node if it is not inside the known table.

Parameters

din_	DIN of the node to locate
timeout	Maximum time to wait for the node to be located (in milliseconds). Default is 1000 ms.

Returns

ERROR_NONE if the node is known, or an error code if it is not.

> **See also**
> pull

lock()

```
const nodestate_t & lock (
            const char * skey,
            unsigned policy_)
```

Set SKEY and security policy for local node

Parameters

skey the security key to set

policy_ the security policy to set

> **Warning**
> Not implemented yet!

Returns

the nodestate_t of the local node after setting the security key and policy.

map() [1/3]

```
daas_error_t map (
            din_t din)
```

Maps a remote node to the local instance. This is used to establish communication with other nodes in the network.

Parameters

din DIN of the node to map

Returns

ERROR_NONE on success, or an error code on failure.

See also
doInit

map() [2/3]

```
daas_error_t map (
            din_t din,
            link_t link,
            const char * suri)
```

Maps a remote node to the local instance. This is used to establish communication with other nodes in the network.

Parameters

din DIN of the node to map

link the communication technology to use:
- **_LINK_INET4**
- **_LINK_BLUETOOTH**
- **and** others as defined in link_t

suri the physical address of the node:
- For **_LINK_INET4:** a text string in the format "IP:PORT" (e.g., "192.168.1.2:4000")
- For **_LINK_BLUETOOTH:** a text string representing the Bluetooth address (e.g., "00:1A:7D:DA:71:13:CHANNEL")

Returns

ERROR_NONE on success, or an error code on failure.

Note
The communication links supported depend on the build configuration and enabled drivers. Those can be listed using the listAvailableDrivers() function.

See also
doInit

map() [3/3]

```
daas_error_t map (
            din_t din,
            link_t link,
            const char * suri,
            const char * skey)
```

Maps a remote node to the local instance. This is used to establish communication with other nodes in the network.

Parameters

din DIN of the node to map

link the communication technology to use:
- **_LINK_INET4**
- **_LINK_BLUETOOTH**
- **and** others as defined in link_t

suri the physical address of the node:
- For **_LINK_INET4:** a text string in the format "IP:PORT" (e.g., "192.168.1.2:4000")
- For **_LINK_BLUETOOTH:** a text string representing the Bluetooth address (e.g., "00:1A:7D:DA:71:13:CHANNEL")

skey the security key for the node. Currently not implemented.

Returns

ERROR_NONE on success, or an error code on failure.

Note
The communication links supported depend on the build configuration and enabled drivers. Those can be listed using the listAvailableDrivers() function.

See also
doInit

pull()

```
daas_error_t pull (
            din_t din,
            DDO ** inboundDDO)
```

Pulls a DDO from a remote node.

Parameters

din DIN of the remote node to pull data from

inboundDDO pointer to a DDO pointer that will hold the pulled DDO

Returns

ERROR_NONE on success, or an error code on failure.

Example:

```
node.doPerform(PERFORM_CORE_THREAD);

DDO* ddo;
while (!exit) {
    uint32_t n_pulls = 0;
    if (node.locate(remoteDIN) == ERROR_NONE && node.availablesPull(remoteDIN, n_pulls) ==
  ERROR_NONE && n_pulls > 0)
    {
        if (node.pull(remoteDIN, &ddo) == ERROR_NONE)
        {
            fprintf(stdout, "Pulled DDO with typeset: %lu, origin: %llu, timestamp: %llu\n",
                    ddo->getTypeset(), ddo->getOrigin(), ddo->getTimestamp());
        }
        else
        {
            fprintf(stderr, "Failed to pull DDO.\n");
        }
    }
}
```

push()

```
daas_error_t push (
            din_t din,
            DDO * outboundDDO)
```

Pushes a DDO to a remote node.

Parameters

din	DIN of the remote node to send data to
outbound↩ DDO	pointer to the DDO to send

Returns

ERROR_NONE on success, or an error code on failure.

Example:

```
DDO ddo(0); // Typeset 0
const char* payload = "Hello world!";
ddo.setPayload(payload, strlen(payload) + 1); // +1 for null terminator
if (node.locate(remoteDIN) == ERROR_NONE && node.push(remoteDIN, &ddo) == ERROR_NONE)
{
    fprintf(stdout, "DDO sent correctly");
}else{
    fprintf(stderr, "Error during push of the DDO");
}
```

receive()

```
uint32_t receive (
            din_t din,
            uint8_t *& inbound)
```

Receives data from a remote node in a real-time session.

Parameters

din DIN of the remote node to receive data from

inbound reference to a variable that will hold the received data

Returns

the size of data received.

> **See also**
> use

received()

```
uint32_t received (
            din_t din)
```

Checks if there is data available from a remote node in a real-time session.

Parameters

din DIN of the remote node to check for data

Returns

the size of data received.

See also
use

remove()

```
daas_error_t remove (
            din_t din)
```

Removes a node from the local instance and cleans up associated resources.

Parameters

din DIN of the node to remove

Returns

ERROR_NONE on success, or an error code on failure.

send()

```
uint32_t send (
            din_t din,
            uint8_t * outbound,
            uint32_t size,
            uint32_t timeout = 1000,
            uint32_t retry = 3)
```

Sends data to a remote node in a real-time session.

Parameters

din DIN of the remote node to send data to

outbound pointer to the data to send

size size of the data to send

timeout maximum time to wait the ACK from the remote node (in milliseconds). If the ACK is not received within the timeout, the function will retry sending the data.

retry number of retries if no ACK is received. Default is 3.

Returns

the size of data sent.

See also
use

sendStatus()

```
daas_error_t sendStatus (
            din_t din)
```

Send the local node's status to a remote node.

Parameters

din DIN of the remote node to send the status to

Returns

ERROR_NONE on success, or an error code on failure.

setAcceptRequestsLevel()

```
void setAcceptRequestsLevel (
            accept_request_policy_t policy_level)
```

Configures whether the local node accepts incoming packets from other nodes, according to the specified trust policy.

The selected policy defines the validation rules applied to incoming requests and can be tuned to enforce stricter security or allow more flexible communication.

Parameters

policy_level The trust policy to apply:

- **trust_none:** Reject all incoming requests from other nodes, including mapped ones.
- **trust_mapped_only:** Accept requests only from mapped nodes.
- **trust_mapped_and_routed:** Accept requests from mapped nodes and from nodes that route through known nodes as intermediate hops.
- **trust_mapped_and_same_network:** Accept requests from mapped nodes and from nodes that belong to the same network (with the same SID), even if they are not mapped. This is the default policy level.
- **trust_mapped_and_routed_and_same_network:** Accept requests from mapped nodes, from nodes that route through known nodes as intermediate hops, and from nodes that belong to the same network (same SID), even if they are not mapped.
- **trust_all:** Accept requests from all DINs, including unknown ones.

Warning

- **trust_mapped_only** rejects all locate requests from nodes that are not mapped. This policy is recommended only for use cases where security is critical and the network topology is strictly controlled.
- **trust_all** allows all incoming requests, including those from unknown DINs. This may introduce security risks, as unauthorized nodes could interact with the local node. Use this policy only in controlled environments or for testing.

Note

The default policy is **trust_mapped_and_same_network**.

Policies that allow routed nodes (**trust_mapped_and_routed**, **trust_mapped_and_routed_and_same_network**, and **trust_all**) may accept nodes that belong to a different network.

During the discovery process, the policy is temporarily set to **trust_all** to allow communication with all nodes. Once discovery completes or times out, the previous policy is automatically restored.

setATSMaxError()

```
void setATSMaxError (
            int32_t error)
```

Set the maximum error allowed for ATS synchronization. This value defines the acceptable time deviation between the local node and remote nodes in the network. The ATS mechanism uses this parameter to ensure that all nodes maintain a consistent time reference within the specified error margin.

> **Note**
> By default, this value is set to **100** milliseconds.

Parameters

error	the maximum error in milliseconds

setDDOPolicy()

```
daas_error_t setDDOPolicy (
            ddo_policy_t policy)
```

Sets the DDO policy for the local node. This policy determines how to manage the unsuccessful sending of DDOs.

Parameters

policy the DDO policy to set (ddo_policy_t)

- **ddo_policy_skip_on_failure:** Skip sending the DDO on failure and deletes it from the sending buffer.
- **ddo_policy_retry_on_failure:** Retry sending the DDO at every doPerform cycle.
- **ddo_policy_exponential_backoff_retry_on_failure:** Retry sending the DDO with exponential backoff on failure. Currently, the backoff starts at 1 minute and doubles with each failure, up to a maximum of 1 hour.

Note

The default policy is **ddo_policy_skip_on_failure**. If the node with mode **ddo_policy_exponential_backoff_retry_on_failure** receives a DDO from the remote node, the retry timer for that DDO is reset and sending will be retried immediately in the next doPerform cycle.

Attention

The retry policies may change the order of DDOs being sent, especially if multiple DDOs are queued for the same remote node. For example, if DDO1 fails to send and is set to retry later, while DDO2 is successfully sent in the next cycle, DDO2 may be sent before DDO1.

Returns

ERROR_NONE on success, or an error code on failure.

setDiscoveryState()

```
void setDiscoveryState (
            discovery_state_t mode)
```

Configures how the node interacts with the network discovery process. Depending on the selected mode, the node can search for existing networks, host new ones, or remain invisible to other nodes.

Parameters

mode

The discovery activation mode:

- **discovery_off:** Completely disables discovery. The node will neither send nor respond to discovery beacons.
- **discovery_sender_only:** The node actively searches for existing networks but will not host or respond to discovery packets from others.
- **discovery_receiver_only:** The node acts as a host, assigning network parameters to others, but will not seek to join external networks.
- **discovery_full:** Enables full bidirectional discovery (both seeking and hosting/joining networks).

> **Note**
> By default, the node is set to **discovery_sender_only** mode upon initialization.

setOptions()

```
daas_error_t setOptions (
            option_t option,
            uint32_t val)
```

Configures specific options for the DaaS API. This function allows fine-tuning of various parameters that affect the behavior of the API.

Parameters

option

The type of option to set (option_type):

- **option_set_ddo_rx_buffer_size:** Sets the maximum size of the receive buffer for DDOs. The value is expressed in bytes. If the specified size is smaller than the minimum required to store a DDO without payload plus its header (75 bytes), the value is automatically increased to this minimum (75 bytes). **Note:** The default buffer size is 1024 bytes.
- **option_set_ddo_tx_buffer_size:** Sets the maximum size of the sending buffer for DDOs. The value is expressed in bytes. If the specified size is smaller than the minimum required to store a DDO without payload plus its header (75 bytes), the value is automatically increased to this minimum (75 bytes). **Note:** The default buffer size is 1024 bytes.
- **option_set_rt_buffer_size:** Sets the maximum size of the transmit and receive buffers used for real-time sessions. The value is expressed in bytes. **Note:** The default buffer size is 1024 bytes.

Warning

When receiving DMEs, the available buffer space is evaluated only after the reception is completed. As a result, a DDO may still be received even if its size exceeds the configured buffer limit.

Returns

ERROR_NONE on success, or an error code on failure.

status()

```
const nodestate_t & status (
            din_t din)
```

Fetches the status of a remote node.

Parameters

din DIN of the remote node to fetch the status from

Returns

the nodestate_t of the remote node.

storeConfiguration()

```
bool storeConfiguration (
            IDepot * storage_interface)
```

Saves the current configuration to the specified storage interface.

Parameters

storage_interface	the interface to save the configuration (e.g., a file system, database, etc.)

Returns

true if the backup was successful, false otherwise.

See also

loadConfiguration

syncNet()

```
const nodestate_t & syncNet (
            din_t din,
            unsigned bubble_time)
```

Set the local system time on remote node din and synchronize ATS

Parameters

din DIN of the remote node to synchronize

bubble_time max error allowed for synchronization in milliseconds

> **Warning**
> Not implemented yet!

Returns

the nodestate_t of the remote node after synchronization.

syncNode()

```
const nodestate_t & syncNode (
            din_t din,
            unsigned timezone)
```

Set the local system time on remote node din and synchronize ATS.

Parameters

din DIN of the remote node to synchronize

timezone the timezone offset in seconds

> **Warning**
> Not implemented yet!

Returns

the nodestate_t of the remote node after synchronization.

unbindNetwork()

```
daas_error_t unbindNetwork ()
```

Unbinds the local node from the current network by closing all open communication channels. The mapping table is preserved, allowing the node to rebind to the same network later without the need to remap previously known nodes.

> **Warning**
> After unbinding, the node may no longer be able to communicate with some nodes of the previous network, depending on their setAcceptRequestsLevel configuration.

Returns

ERROR_NONE on success, or an error code on failure.

unlock()

```
const nodestate_t & unlock (
            din_t din,
            const char * skey)
```

Unlocks a remote node by setting its security key.

Parameters

din	DIN of the remote node to unlock
skey	the security key to set

> **Warning**
> Not implemented yet!

Returns

the nodestate_t of the remote node after unlocking.

use()

```
bool use (
            din_t din,
            uint32_t rx_queue_size = 0,
            uint32_t retry = 3,
            uint32_t timeout = 1000)
```

Starts a real-time session with a remote node. This function establishes a real-time communication channel with the specified remote node. All transmitted data must be acknowledged (ACK) by the remote node. If the ACK is not received within the specified timeout, the function retries sending the data for a configured number of attempts.

Parameters

din — The DIN of the remote node with which to start the session.

rx_queue_size — The size of the receive queue for incoming data from the remote node. If set to 0, a default size of 1024 will be used.

retry — The number of retries if no ACK is received. Default is 3.

timeout — The maximum time to wait for an ACK from the remote node (in milliseconds). If the ACK is not received within this time, the function will retry sending the data.

Returns

true if the real-time session is successfully established and the connection is opened; false otherwise.

Example:

```
class event_daas : public IDaasApiEvent {
    public:

    // Other event handlers...

    // Triggered after stream information is received
    void streamInfoReceived(din_t din, stream_type pkt_type, uint32_t stream_id) override {
        switch (pkt_type)
        {

        case stream_type::STREAM_OPEN:
            printf("Stream open request received from DIN %llu for stream ID %u\n", din,
      stream_id);
            break;

        case stream_type::STREAM_DATA:
        {
            uint32_t block_size = node.received(din);
            if (block_size > 0) {
                uint8_t *buffer;
                uint32_t read_size = node.receive(din, buffer);
                printf("Received stream data: size=%u, data=%.*s\n", read_size, read_size,
      buffer);
            }
            break;
        }

        case stream_type::STREAM_CLOSE:
            printf("Stream close request received from DIN %llu for stream ID %u\n", din,
      stream_id);
            break;

        default:
            break;
        }
    }
};

bool exit = false;
DaasAPI node = DaasAPI(new event_daas(), "MACOS");

int main() {

   // Init logic

    // To send data using the stream functions
    din_t remoteDIN = 201; // DIN of the remote node to send data to
    uint8_t *buffer = (uint8_t*)"Hello, this is a stream test!";

    // Open the stream
    node.use(remoteDIN);

    // Send data over the stream
    for (int i = 0; i < 10; i++){
        int sent = node.send(remoteDIN, buffer, strlen((char*)buffer) + 1);
        printf("Sent %d bytes\n", sent);
    }

    // Close the stream
    node.end(remoteDIN);

    return 0;
}
```

1.2 IDaasApiEvent Class Reference

`#include <daas_types.hpp>`

Public Member Functions

- virtual void dinAccepted (din_t)=0
- virtual void ddoReceived (int payload_size, typeset_t, din_t)=0
- virtual void frisbeeReceived (din_t)=0

- virtual void nodeStateReceived (din_t)=0
- virtual void atsSyncCompleted (din_t)=0
- virtual void frisbeeDperfCompleted (din_t, uint32_t packets_sent, uint32_t block_size)=0
- virtual void nodeDiscovered (din_t din, link_t link)=0
- virtual void nodeConnectedToNetwork (din_t sid, din_t din)=0
- virtual void streamInfoReceived (din_t din, stream_type pkt_type, uint32_t stream_id)=0

1.2.1 Detailed Description

Interface used to implement event handlers for DaaS API events. The functions are called by the DaaS API library when specific events occur, allowing the user to define custom behavior in response to these events.

1.2.2 Member Function Documentation

atsSyncCompleted()

```
virtual void atsSyncCompleted (
            din_t )  [pure virtual]
```

Called when ATS synchronization is completed.

Parameters

din The din of the node that has completed ATS synchronization.

ddoReceived()

```
virtual void ddoReceived (
            int payload_size,
            typeset_t ,
            din_t )  [pure virtual]
```

Called when a new DDO is received.

Parameters

payload_size	Size of the received payload.
typeset	Typeset of the received DDO.
din	DIN of the sender node.

dinAccepted()

```
virtual void dinAccepted (
            din_t )  [pure virtual]
```

Called when a din is accepted by the node (incoming request).

Parameters

din	The din that has been accepted.

frisbeeDperfCompleted()

```
virtual void frisbeeDperfCompleted (
            din_t ,
            uint32_t packets_sent,
            uint32_t block_size)  [pure virtual]
```

Called when a frisbee dperf operation is completed.

Parameters

din	The din of the node that completed the frisbee dperf.
packets_sent	Number of packets sent during the dperf operation.
block_size	Size of each data block sent.

> **Note**
> To get detailed results, refer to the dperf_info_result structure returned by getFrisbeeResultDPERF().

frisbeeReceived()

```
virtual void frisbeeReceived (
            din_t )  [pure virtual]
```

Called when a frisbee message is received.

Parameters

din	The din of the node that sent the frisbee message.

nodeConnectedToNetwork()

```
virtual void nodeConnectedToNetwork (
            din_t sid,
            din_t din)  [pure virtual]
```

Called when this node connects to a DaaS network.

Parameters

sid The SID of the connected network.

din The update DIN of this node used inside the network.

nodeDiscovered()

```
virtual void nodeDiscovered (
            din_t din,
            link_t link)  [pure virtual]
```

Called when a new node is discovered in the network.

Parameters

din The din of the newly discovered node.

link The link through which the node was discovered.

> **Warning**
> The node might have not be accepted yet, this event is just to inform that a new node has been seen.

nodeStateReceived()

```
virtual void nodeStateReceived (
            din_t )  [pure virtual]
```

Called when a node state message is received.

Parameters

din	The din of the node whose state has been received.

streamInfoReceived()

```
virtual void streamInfoReceived (
            din_t din,
            stream_type pkt_type,
            uint32_t stream_id)  [pure virtual]
```

Called when a stream information is received.

Parameters

din The din of the node that sent the stream information.

pkt_type The type of the packet received: (1) - STREAM_OPEN: indicates that a stream open request has been received. (2) - STREAM_DATA: indicates that a stream data packet has been received. (3) - STREAM_CLOSE: indicates that a stream close request has been received.

stream_id The ID of the stream associated with the received packet.

1.3 nodestate_t Struct Reference